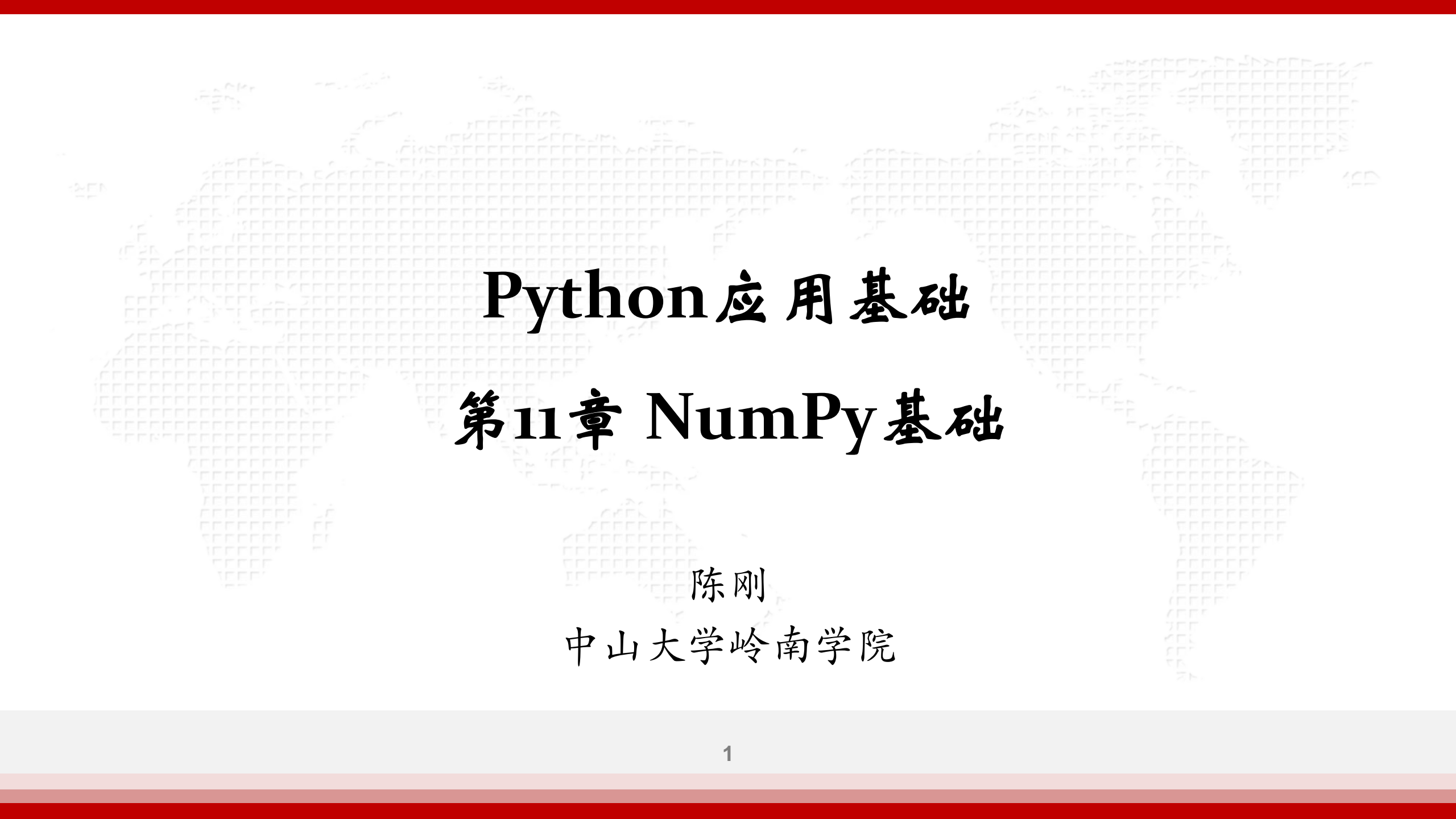




Python应用基础

第11章 NumPy基础

陈刚

中山大学岭南学院

数据分析篇目录

Contents

第 11 章

NumPy 基础

第 12 章

Pandas 金融数据分析

第 13 章

Matplotlib 数据可视化

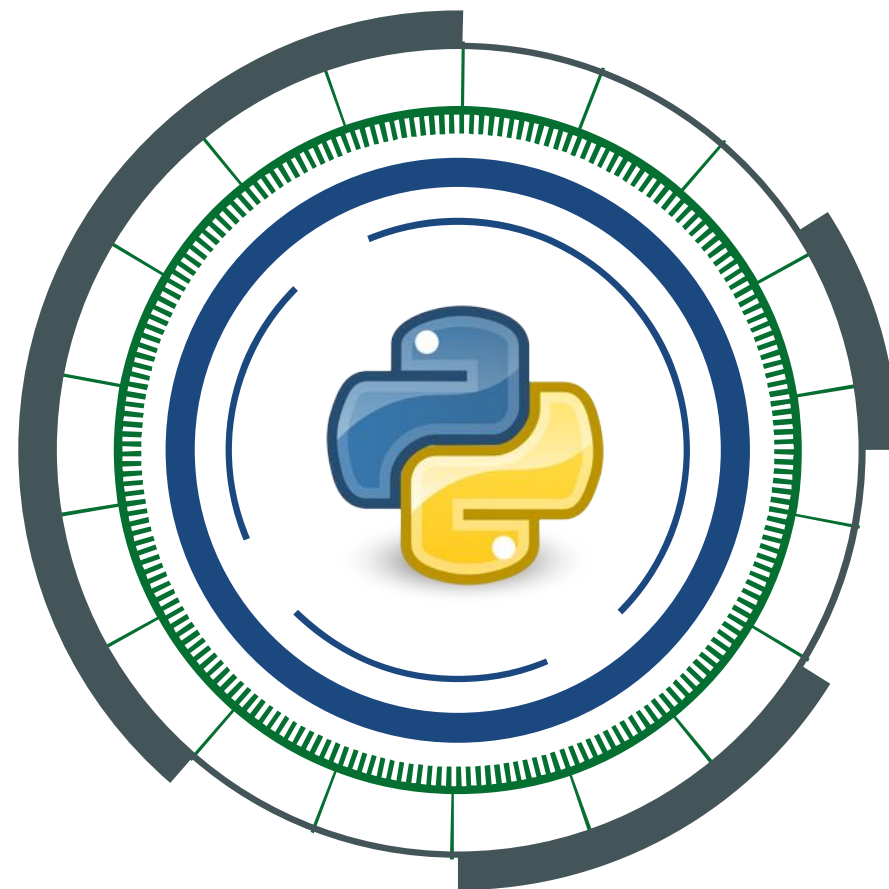


pythonTM
programming



学习目标

- 1 掌握多维数组对象 ndarray 的使用方法
- 2 掌握 NumPy 数组元素访问的方法
- 3 掌握 NumPy 数组中元素排序的方法
- 4 掌握 NumPy 数组中元素结构重塑的方法
- 5 掌握 NumPy 数组运算方法
- 6 熟悉 Numpy 通用函数





11.1

多维数组对象 NDARRAY

概述

- NumPy (Numerical Python) 是 Python 进行数值计算最重要的三方模块。
- NumPy 的大部分代码是用 C 或者 Fortran 实现的，运行速度比纯 Python 代码要快。
- NumPy 不需要使用 for 或者 while 循环就可以完成整个数组的运算。

多维数组对象 ndarray

- NumPy 中利用一种 N 维数组对象 ndarray 来存储和处理数据。
- 该对象可以定义维度， 适合做代数运算。
- ndarray 是标量元素之间的运算。

多维数组对象 ndarray

【例】 已知人民币兑 100 外币的外汇牌价汇率为：美元 680.6、欧元 800.05、英镑 882.6 和港币 87.79。现有 10 万元人民币，计算可以兑换的外币数量。

```
❶ exchange_rate = [680.6, 800.05, 882.6, 87.79]
❷ for rate in exchange_rate:
❸     print(round((100000 * 100) / rate), end=' ')
14693 12499 11330 113908
```

```
❹ import numpy as np
❺ rate_array = np.array([680.6, 800.05, 882.6, 87.79])
❻ np.round((100000 * 100) / rate_array)
array([ 14693., 12499., 11330., 113908.])
```

创建 ndarray 数组

- `array()` 方法，将列表、元组、数组或其他序列类型转换为 ndarray 数组。

```
① int_array = np.array([1, 2, 3, 4, 5])
```

```
② int_array  
array([1, 2, 3, 4, 5])
```

```
③ int_array.dtype
```

```
dtype('int64')
```

```
④ float_array = np.array([1, 2, 3, 4, 5], dtype=np.float64)
```

```
⑤ float_array
```

```
array([1., 2., 3., 4., 5.])
```

```
⑥ float_array.dtype
```

```
dtype('float64')
```


创建 ndarray 数组

- `arange()` 方法，与 `range()` 函数类似，在 NumPy 中，通过 `arange()` 方法可以得到一个 NumPy 数组。

```
arange([start,] stop[, step,], dtype=None)
```

- `start` 是可选参数，代表起始值，默认值为 0
- `stop` 是截止值，结果中并不包含该值
- `step` 是可选参数，设置的是步长
- `dtype` 设置数据类型，默认为 `None` 值
- NumPy 将推测数组的数据类型。

```
① np.arange(5)
```

```
array([0, 1, 2, 3, 4])
```

```
② np.arange(5.0)
```

```
array([0., 1., 2., 3., 4.])
```

```
③ np.arange(2, 5.0)
```

```
array([1, 2, 3, 4, 5])
```

```
④ np.arange(2, 10, 2)
```

```
array([2, 4, 6, 8])
```

创建 ndarray 数组

● ones()、zeros()、empty() 和 full() 方法，均可以接受一个元组作为数组的形状且 dtype 指定数据类型。

- ones() 方法：产生全 1 数组
- zeros() 方法：产生全 0 数组
- empty() 方法：产生新数组，只分配空间不填充任何值。
- full() 方法：利用指定值填充产生的数组。

① `np.ones((2, 3))`

```
array([[1., 1., 1.],  
       [1., 1., 1.]])
```

② `np.zeros((2, 3), dtype=np.int32)`

```
array([[0, 0, 0],  
       [0, 0, 0]], dtype=int32)
```

③ `np.empty((2, 2))`

```
array([[1.e-323, 2.e-323],  
       [3.e-323, 4.e-323]])
```

④ `np.full((2, 2), 5)`

```
array([[5, 5],  
       [5, 5]])
```

ndarray 数组的属性

- dtype 属性，在 NumPy 数组中，要求所有元素的数据类型是一致的。常用的 NumPy 数据类型如表所示。

类型	类型代码	说明
int8、uint8	i1、u1	有符号、无符号 8 位整型
int16、uint16	i2、u2	有符号、无符号 16 位整型
int32、uint32	i4、u4	有符号、无符号 32 位整型
int64、uint64	i8、u8	有符号、无符号 64 位整型
float32	f4	标准单精度浮点数
float64	f8	标准双精度浮点数
bool		布尔类型
object		Python 对象类型
unicode_	U	固定长度 unicode 类型，如 U4

```
❶ rate_array = np.array([680.6, 800, 882.6, 87.79])
❷ rate_array.dtype
dtype('float64')
❸ rate_array.astype(np.int32)
array([680, 800, 882, 87], dtype=int32)
```

这些类型是 NumPy 中的数据类型，因此在使用时需要使用 NumPy 类来引用。例如 np.int32，其中 np 是 NumPy 的别名。

ndarray 数组的属性

- `ndim` 属性，NumPy 数组可以是多维的。`ndim` 属性用来查看数组的维数。

```
❶ n_array = np.ones((3,4))  
❷ n_array  
array([[1., 1., 1., 1.],  
       [1., 1., 1., 1.],  
       [1., 1., 1., 1.]])  
❸ n_array.ndim  
2
```

- `shape` 属性，通过 `shape` 属性得到的是数组的形状。

```
❹ n_array.shape  
(3, 4)
```



11.2

访问数组元素

普通索引

- 与 Python 内置列表相似，可以通过下标索引（整数索引）的方式获取 NumPy

数组中的元素：

```
❶ close_price = np.array([[3.22, 3.21, 3.20, 3.19, 3.20],  
                           [3.19, 3.16, 3.17, 3.15, 3.15],  
                           [4.96, 4.93, 4.92, 4.90, 4.89],  
                           [6.21, 6.19, 6.13, 6.17, 6.20]])  
  
❷ close_price[0][1]  
3.21
```

- NumPy 对索引进行了扩展，可以通过将每个维度的索引值列在中括号中的方式来获取对应位置的元素：

```
❸ close_price[0, 1]  
3.21
```

切片

- 切片用于获取数组中的一小块数据，其表示形式和 Python 内置列表的切片相似：在中括号中用冒号连接给出切片起始位置、截止位置（不包含）和步长。

```
❶ close_price[0:2]
   array([[3.22, 3.21, 3.2, 3.19, 3.2],
          [3.19, 3.16, 3.17, 3.15, 3.15]])

❷ close_price[:2]
   array([[3.22, 3.21, 3.2, 3.19, 3.2],
          [3.19, 3.16, 3.17, 3.15, 3.15]])

❸ close_price[0:2, 0::2]
   array([[3.22, 3.2, 3.2],
          [3.19, 3.17, 3.15]])
```

切片

- 在获取多维数据的情景下，NumPy 的切片更加强大。

```
❶ close_price[0:3, 1:3]  
array([[3.21, 3.2 ],  
       [3.16, 3.17],  
       [4.93, 4.9 ]])
```

- 整数索引和切片可以混合使用，得到低维度的切片。

```
❶ slice_close_price = close_price[1:, -1]  
slice_close_price  
array([3.15, 3.2, 6.2])  
  
❷ slice_close_price.ndim  
1
```


切片

- 在对切片赋值时，可以将一个标量值赋给切片，这样的赋值操作会被扩散到整个切片 区域：

```
❶ close_price[:, -2] = 0
close_price
array([[3.22, 3.21, 3.2, 0. , 3.2],
       [3.19, 3.16, 3.17, 0. , 3.15],
       [4.96, 4.93, 4.92, 0. , 4.89],
       [6.21, 6.19, 6.13, 0. , 6.2]])
```

切片

- 当然，也可以使用数组或列表对切片赋值。数组或列表的形状与切片的形状一致或者可广播（广播指的是不同形状的 NumPy 数组之间运算的执行方式，是 NumPy 数组非常强大的功能）。

```
❶ close_price[:, -2] = [3.19, 3.15, 4.9, 6.17]
close_price
array([[3.22, 3.21, 3.2 , 3.19, 3.2 ],
       [3.19, 3.16, 3.17, 3.15, 3.15],
       [4.96, 4.93, 4.92, 4.9 , 4.89],
       [6.21, 6.19, 6.13, 6.17, 6.2 ]])
```

布尔索引

- NumPy 数组的比较运算是矢量化的，即数组和标量进行比较时，得到的是数组中每个元素与标量比较结果组成的布尔数组。

```
❶ bank_array = np.array(['中国银行', '农业银行', '工商银行', '建设银行'])  
❷ bank_array == '农业银行'  
array([False, True, False, False])
```

- 可以将这个布尔型的数组用于数组索引。

```
❸ close_price[bank_array=='农业银行']  
array([[3.19, 3.16, 3.17, 3.15, 3.15]])
```

布尔索引

- 可以使用 &（和）、|（或）之类的布尔运算符组合多个条件。

```
❶ close_price[(bank_array=='中国银行') | (bank_array=='工商银行')]  
  
array([[3.22, 3.21, 3.2 , 3.19, 3.2 ],  
       [4.96, 4.93, 4.92, 4.9 , 4.89]])
```

- 布尔索引也可以用于设置数组中特定元素的值。

```
❷ close_price[0,1] = -3.21  
❸ close_price[3,3] = -6.17  
❹ close_price[close_price < 0] = 0  
❺ close_price  
  
array([[3.22, 0. , 3.2 , 3.19, 3.2 ],  
       [3.19, 3.16, 3.17, 3.15, 3.15],  
       [4.96, 4.93, 4.92, 4.9 , 4.89],  
       [6.21, 6.19, 6.13, 0. , 6.2 ]])
```

花式索引

- 花式索引是利用整数列表对数据进行索引。只需要传入一个用于指定顺序的整数列表或 ndarray 数组，就可以得到按照特定顺序选取的子集。

```
❶ bank_array = np.array(['中国银行', '农业银行', '工商银行', '建设银行'])  
❷ bank_array[[3,2,1]]  
  
array(['建设银行', '工商银行', '农业银行'], dtype='<U4')
```



11.3

排序

sort() 排序

- 该方法得到值排序后的结果。对于多维数组，NumPy 允许通过 axis 参数指定按照特定维度（轴）进行排序：axis=0 代表按列对元素进行排序，axis=1 代表按行排序，不输入参数时，默认按行排序。

【例】 利用第 11.2 节中的股票收盘价数据，找出中国银行、农业银行、工商银行和建设银行股票在 2020 年 9 月 21 日至 9 月 25 日的最低收盘价和最高收盘价。

```
① close_price = np.array([[3.22, 3.21, 3.20, 3.19, 3.20],  
                           [3.19, 3.16, 3.17, 3.15, 3.15],  
                           [4.96, 4.93, 4.92, 4.90, 4.89],  
                           [6.21, 6.19, 6.13, 6.17, 6.20]])  
  
② close_price.sort()  
③ close_price[:,0] # 四家银行的最低收盘价  
array([3.19, 3.15, 4.89, 6.13])  
  
④ close_price[:, -1] # 四家银行的最高收盘价  
array([3.22, 3.19, 4.96, 6.21])
```

argsort() 排序

- NumPy 数组的 `argsort()` 方法返回的是按照元素排序后，元素在原数组中的下标列表。该方法同样可以使用 `axis` 参数改变排序规则。

【例】 利用前例的股票收盘价数据，将中国银行、农业银行、工商银行和建设银行按 2020 年 9 月 22 日收盘价进行排序。

```
① close_price = np.array([[3.22, 3.21, 3.20, 3.19, 3.20],
                           [3.19, 3.16, 3.17, 3.15, 3.15],
                           [4.96, 4.93, 4.92, 4.90, 4.89],
                           [6.21, 6.19, 6.13, 6.17, 6.20]])

② bank_array = np.array(['中国银行', '农业银行', '工商银行', '建设银行'])
③ sorted_index = close_price.argsort(axis=0)
   sorted_index
   array([[1, 1, 1, 1, 1],
          [0, 0, 0, 0, 0],
          [2, 2, 2, 2, 2],

④ bank_array[sorted_index[:,1]]
   array(['农业银行', '中国银行', '工商银行', '建设银行'], dtype='<U4')
```




11.4

数组重塑

resize() 和 reshape() 方法

- 通过传入新的形状，`resize()` 和 `reshape()` 方法都可以改变数组的形状。不同之处在于：`resize()` 方法是进行“就地修改”（即改变了数组本身），`reshape()` 方法则是生成新的数组。

```
❶ close_price_list = [3.22, 3.21, 3.20, 3.19, 3.20, 3.19, 3.16, 3.17, 3.15,  
3.15, 4.96, 4.93, 4.92, 4.90, 4.89, 6.21, 6.19, 6.13, 6.17, 6.20]  
❷ close_price_array = np.array(close_price_list)  
❸ close_price_array
```

```
array([3.22, 3.21, 3.2, 3.19, 3.2, 3.19, 3.16, 3.17, 3.15, 3.15, 4.96, 4.93, 4.92, 4.9, 4.89, 6.21, 6.19, 6.13, 6.17, 6.2 ])
```

```
❹ close_price_array.shape  
(20,)
```

```
❺ close_price_array.reshape((4,5))  
array([[3.22, 3.21, 3.2, 3.19, 3.2], [3.19, 3.16, 3.17, 3.15, 3.15], [4.96, 4.93, 4.92, 4.9, 4.89], [6.21, 6.19, 6.13, 6.17, 6.2 ]])
```

```
❻ close_price_array.resize((4,5))  
❼ close_price_array
```

```
array([[3.22, 3.21, 3.2, 3.19, 3.2], [3.19, 3.16, 3.17, 3.15, 3.15], [4.96, 4.93, 4.92, 4.9, 4.89], [6.21, 6.19, 6.13, 6.17, 6.2 ]])
```

transpose() 和 swapaxes() 方法

- 对于多维数组，使用 `transpose()` 方法来转换维度，产生新的数组。该方法不带参数是对数组进行转置，也可以传入新的维度。例如，对于二维来说，`transpose()` 和 `transpose(1, 0)` 的结果是一样的。

```
❶ close_price = np.array([[3.22, 3.21, 3.20, 3.19, 3.20],
                           [3.19, 3.16, 3.17, 3.15, 3.15],
                           [4.96, 4.93, 4.92, 4.90, 4.89],
                           [6.21, 6.19, 6.13, 6.17, 6.20]])
❷ close_price.transpose() # 或者close_price.transpose(1, 0)

array([[3.22, 3.19, 4.96, 6.21],
       [3.21, 3.16, 4.93, 6.19],
       [3.2 , 3.17, 4.92, 6.13],
       [3.19, 3.15, 4.9 , 6.17],
       [3.2 , 3.15, 4.89, 6.2 ]])
```

transpose() 和 swapaxes() 方法

- `swapaxes()` 方法通常用于维度交换，在参数中仅传入一对需要交换的维度编号。

```
❶ close_price = np.array([[3.22, 3.21, 3.20, 3.19, 3.20],  
                           [3.19, 3.16, 3.17, 3.15, 3.15],  
                           [4.96, 4.93, 4.92, 4.90, 4.89],  
                           [6.21, 6.19, 6.13, 6.17, 6.20]])  
❷ close_price.swapaxes(0,1)  
  
array([[3.22, 3.19, 4.96, 6.21],  
       [3.21, 3.16, 4.93, 6.19],  
       [3.2 , 3.17, 4.92, 6.13],  
       [3.19, 3.15, 4.9 , 6.17],  
       [3.2 , 3.15, 4.89, 6.2 ]])
```

flatten() 方法

- `flatten()` 方法返回一个折叠成一维的数组。看上去像是 `reshape()` 方法的反向操作。

```
❶ close_price = np.array([ [3.22, 3.21, 3.20, 3.19, 3.20],  
                           [3.19, 3.16, 3.17, 3.15, 3.15],  
                           [4.96, 4.93, 4.92, 4.90, 4.89],  
                           [6.21, 6.19, 6.13, 6.17, 6.20] ] )  
  
❷ close_price.flatten()  
  
array([3.22, 3.21, 3.2 , 3.19, 3.2 , 3.19, 3.16, 3.17, 3.15, 3.15, 4.96, 4.93, 4.92,  
       4.9 , 4.89, 6.21, 6.19, 6.13, 6.17, 6.2 ])
```



11.5

NUMPY 数组间运算

NumPy 数组间运算

- NumPy 数组的矢量化除了比较，更多的是应用于批量运算。对于形状相同的数组之间的算术运算也会应用到元素级。

【例】股票涨跌幅是对涨跌值的描述，计算公式为：

$$\text{涨跌幅} = (\text{当前交易日收盘价} - \text{前一交易日收盘价}) / \text{前一交易日收盘价}$$

利用前例股票收盘价数据，计算银行股票 2020 年 9 月 22 日至 9 月 25 日的价格涨跌幅。

```
① close_price = np.array([[3.22, 3.21, 3.20, 3.19, 3.20],
                           [3.19, 3.16, 3.17, 3.15, 3.15],
                           [4.96, 4.93, 4.92, 4.90, 4.89],
                           [6.21, 6.19, 6.13, 6.17, 6.20]])
② close_price1 = close_price[:,1:]
③ close_price2 = close_price[:, :-1]
④ np.round((close_price1 - close_price2) / close_price2, 4)

array([[ -0.0031, -0.0031, -0.0031,  0.0031],
       [-0.0094,  0.0032, -0.0063,  0. ],
       [-0.006 , -0.002 , -0.0041, -0.002 ],
       [-0.0032, -0.0097,  0.0065,  0.0049]])
```



11.6

UFUNC() 通用函数

ufunc() 通用函数

- NumPy 中有一种对 ndarray 数组中数据执行元素级运算的函数，称为通用函数。利用通用函数可以不需要使用循环而快速地进行元素级的运算。注意，这些函数是 NumPy 模块中的函数，因此调用时，需要使用“np. 函数名 ()”或者“NumPy 数组. 函数名 ()”的形式。

函数	说明
sum	对数组内部元素进行求和运算
mean	对数组内部元素进行求均值运算
prod	对数组内部元素进行求乘积运算
max、min	求数组内部元素最大值、最小值
abs	计算整数、浮点数的绝对值
sqrt	计算数组内部元素个平方根
exp	计算数组内部元素的指数 e^x
ceil	计算大于等于数组内部相应元素的最小整数
floor	计算小于等于数组内部相应元素的最大整数
round()	得到每个元素的四舍五入值

ufunc() 通用函数

【例】 计算前面案例中每只股票收盘价的平均值。

```
❶ close_price = np.array([[3.22, 3.21, 3.20, 3.19, 3.20],  
                           [3.19, 3.16, 3.17, 3.15, 3.15],  
                           [4.96, 4.93, 4.92, 4.90, 4.89],  
                           [6.21, 6.19, 6.13, 6.17, 6.20]])  
❷ np.mean(close_price, axis=1)  
   array([3.204, 3.164, 4.92 , 6.18 ])  
❸ close_price.mean(axis=1)  
   array([3.204, 3.164, 4.92 , 6.18 ])
```

Happy coding! 🐍

